

Weekly Project Progress-4

Group Work

Oussema Kirmani – 5153785

Gurpreet Singh Galsian– 5154655

Dhruv Dewan – 5150667

Data Analysis with Python

Algoma University

COSC-5806

Zamilur Rahman

March 29, 2026

In the fourth weekly progress, we have used the two models to predict the target variable(heart disease):

1. Random Forest Classifier: It is an ensemble learning method that builds numerous decision trees during training and outputs the majority vote of the classes (classification).
2. XGB Classifier: It is a powerful machine learning model from the [XGBoost \(eXtreme Gradient Boosting\)](#) library specifically designed for classification tasks. It improves continuously by mistakes in the previous tree.

We dropped 65000 samples with a 0 value in the heart disease column to balance the data. So, our model will not be biased.

We implemented the one-hot encoding to convert the categorical features ("Sex", "Chest pain type", "Exercise angina", "Slope of ST") to numbers (0 and 1) so that machine learning models can understand them.

Then we combined the numerical and categorical features in x and y, which will be our output column. It will predict whether a person has heart disease or not.

Moreover, we have split the training and test data with `train_test_split`. In which we divided it into four parts:

- a) `x_train`: Input data for learning
- b) `y_train`: Correct answers for learning
- c) `x_test` : New unseen data
- d) `y_test`: Real answers (for checking)

We implemented stratified sampling to keep the same proportion of classes (0 and 1) in both train. Test and train data are divided in 20% and 80%, respectively, with `random state = 42`, which is used to control the randomness to make the results repeatable and consistent.

Then we normalized the data with a standard scalar to make all numerical values similar in scale with the formula:

New value = (Original value – Average) ÷ Spread.

Furthermore, Model Learnt Average, Spread from data, and scaled the numerical features of X_train with fit_transform. Then, scaled X_test with “transform” as our model already knows the average and Spread value from the data.]

Random Forest

Imported the RandomForestClassifier from sklearn.ensemble

We used 200 trees with n_estimators=200 and random_state=42. After that, our model learns the pattern from X_train (features) and Y_train (correct answers) using fit.

Afterwards, we checked “Did our model learn correctly or not?”

1. We provided unseen data “X_test” to the model to predict “Y_pred”, and after that, we checked the prediction " Y_pred " with y_test, which are the real answers.
2. We imported accuracy_score and classification_report to measure the performance of the model.
3. We found that our model predicted 0.8773008849557522 accurately.
4. In the classification report, we checked the detailed performance of each class.

Output contains:

Metric	Meaning
Precision	When the model says disease, how often is it correct?
Recall	Out of all real disease cases, how many did the model find?
F1-score	Balance of precision & recall
Support	Number of samples

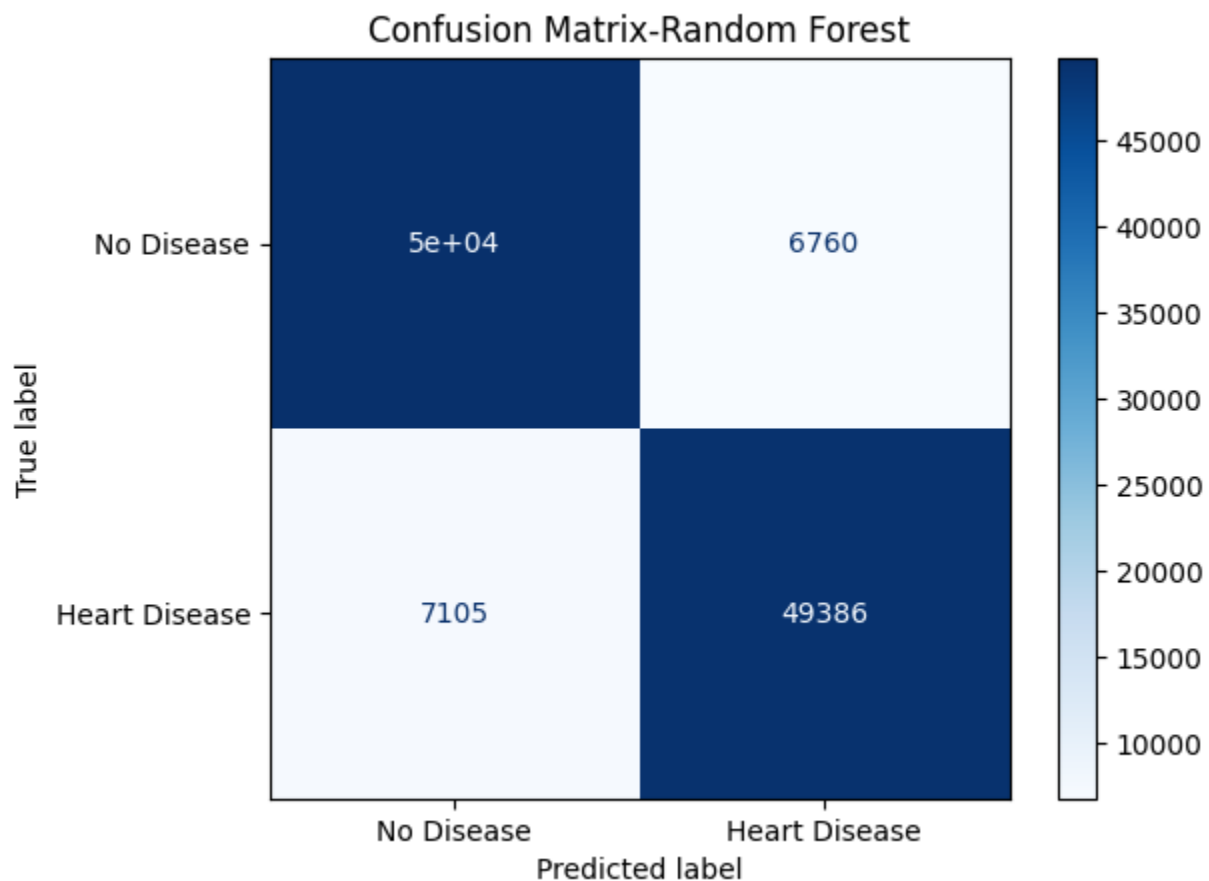
	precision	recall	f1-score	support
No Disease	0.88	0.88	0.88	56509
Heart Disease	0.88	0.87	0.88	56491
accuracy			0.88	113000
macro avg	0.88	0.88	0.88	113000
weighted avg	0.88	0.88	0.88	113000

5. We used the confusion matrix to check how many predictions are correct with confusion_matrix and visualize it with ConfusionMatrixDisplay, where y_test is the real answers and y_pred are the model predictions, then compare both and create a table

```
[[TN FP]
 [FN TP]]
```

Meaning of each value

TN (Top-left)	True Negative, Correctly predicted No Disease
FP (Top-right)	False Positive, Predicted Disease, but actually No Disease
FN (Bottom-left)	False Negative, Predicted No Disease, but actually had a disease
TP (Bottom-right)	True Positive, Correctly predicted Disease



Explanation of the confusion matrix:

- Top Left (**True Negative (TN)**) shows the model predicted 50,000 patients have no disease, and actually they do not have disease, which means a correct prediction.
- Top Right (**False Positive (FP)**) shows the model predicted 6,760 patients have disease, but actually they do not have disease, which is a false alarm

- c) Bottom-left (**False Positive (FP)**) shows the model predicted 7,105 patients have no disease, but actually they have the disease
- d) Bottom-right (**True Positive (TP)**) shows the model predicted 49,386 patients have disease, and actually they have disease, which shows the correct prediction.

Summary of Random Forest Model:

Total Correct prediction: 99,386

Total Wrong prediction: 13,865

With the Random Forest Model, our model is predicting with 87% accuracy.

XGBoost Model

Imported XGBClassifier from xgboost, then we selected the numerical and categorical features. Furthermore, we have converted the categorical column into binary columns using one-hot encoding, as we did in the random forest model. After that, we combined numerical and categorical feature then divided the data using train_test_split with stratified sampling. Before training the data, we scaled the data using StandardScaler().

1. We created the machine learning model using XGBClassifier()
2. We used 350 trees using n_estimators where more trees means better learning.
3. We implemented the learning speed with learning_rate = 0.1 (which means How fast the model learns from mistakes)

Value	Meaning
High (0.5)	Learns fast but may make mistakes
Low (0.1)	Learns slowly but carefully(safe and stable)

4. We defined the depth of each tree with max_depth = 5 which determine how complex each tree is.

Depth	Meaning
Small	Simple model
Large	Complex model (may overfit)

As we used max_depth = 5 which means balanced (not too simple, not too complex)

5. Our model measures the error with `eval_metric = 'logloss'` where lower logloss indicates better predictions.
6. Model is learning from data using `xgb_model.fit(X_train, y_train)`

Now our model is trained and it will predict the output on unseen data based on what learned using `y_pred = xgb_model.predict(X_test)`

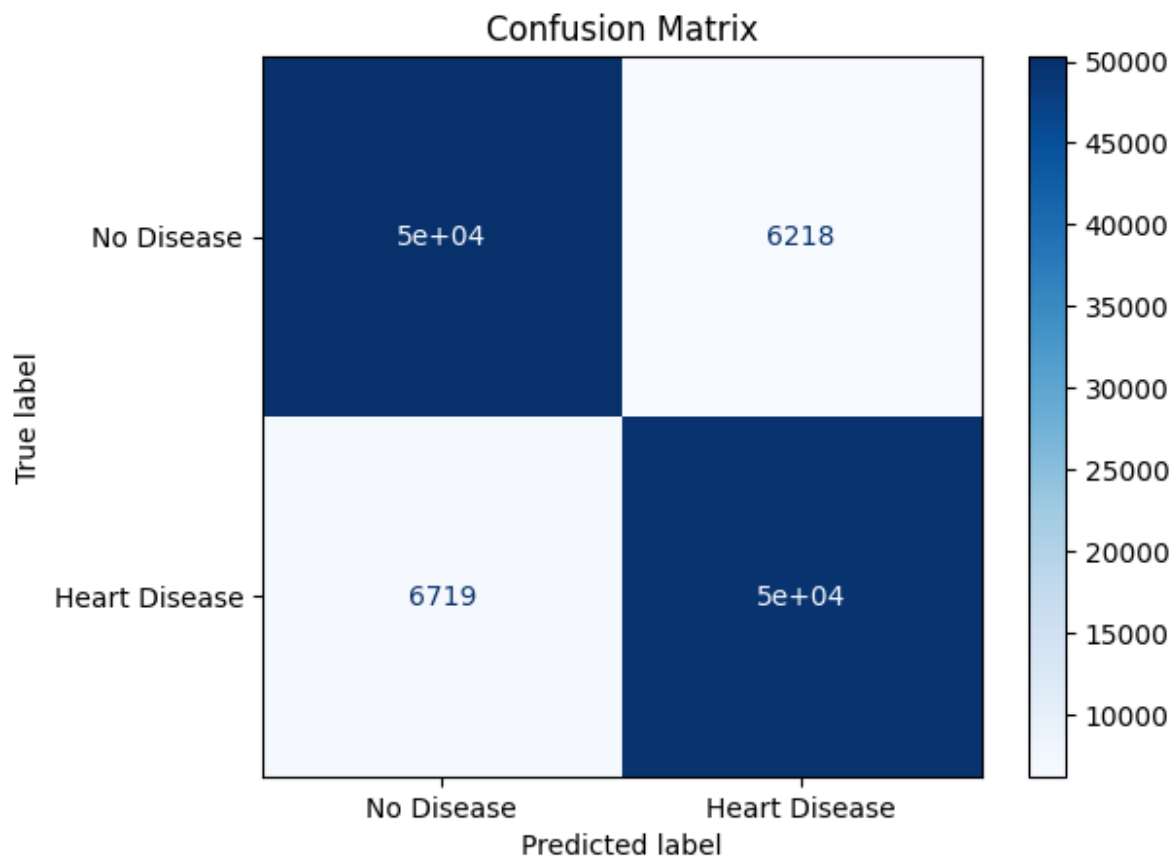
Then our model predicted the probability instead of 0 & 1 with `y_prob = xgb_model.predict_proba(X_test)[: ,1]`

Where, `predict_proba()` gives 2 columns and `[: ,1]` provide only probability of class 1.

We used `accuracy_score` and `classification_report` to measure the performance of the model.

With XGBoost model, predictions are 0.8855 accurate

Classification Report:					
	precision	recall	f1-score	support	
No Disease	0.88	0.89	0.89	56509	
Heart Disease	0.89	0.88	0.88	56491	
accuracy			0.89	113000	
macro avg	0.89	0.89	0.89	113000	
weighted avg	0.89	0.89	0.89	113000	



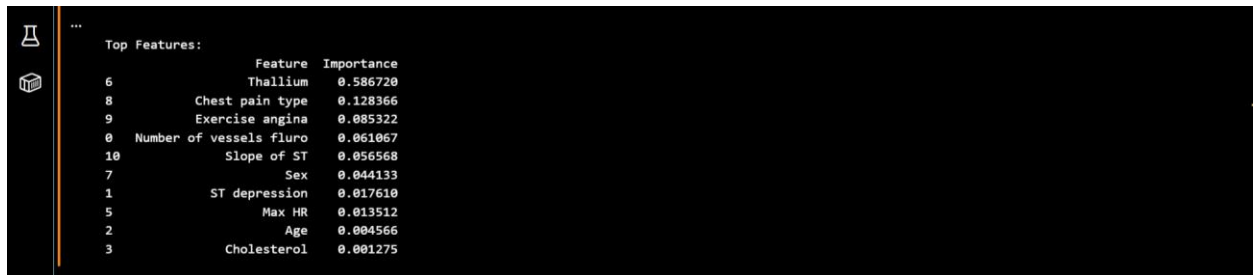
Summary of XGBoost Model:

Total Correct prediction: 100,000

Total Wrong prediction: 13,000

With the XGBoost Model, our model is predicting with 88% accuracy.

Finally , we demonstrate the most important to least important feature which helped in prediction.



	Feature	Importance
6	Thallium	0.586720
8	Chest pain type	0.128366
9	Exercise angina	0.085322
0	Number of vessels fluoro	0.061067
10	Slope of ST	0.056568
7	Sex	0.044133
1	ST depression	0.017610
5	Max HR	0.013512
2	Age	0.004566
3	Cholesterol	0.001275